

give me an example of a case where built in javascript sort method even with necessary parameters like comparator function is not applicable and have to use one of sorting algorithms if there is such a case

Here's an explanation:

Even though JavaScript's built-in `sort()` method with a custom comparator is very flexible for many cases, there are situations where it isn't applicable. One such case is when you're working with data that is too large to all fit into memory at once.

For example, imagine you have a dataset that spans multiple gigabytes or even terabytes of data stored on disk. The built-in `sort()` method requires loading the entire array into memory before sorting, which might not be feasible or efficient for extremely large datasets. In these scenarios, you need to use an external sorting algorithm, such as external merge sort, which works by splitting the data into smaller chunks that can be individually sorted (possibly using the built-in `sort` or another efficient algorithm) and then merged together. This approach is necessary because it allows sorting data that exceeds available memory by reading and writing portions of the data from disk.

Another example could be a case where you need a parallelized sorting approach to leverage multi-threading in a way that the built-in `sort()` cannot. For very large datasets or specialized applications requiring concurrent processing, you'd implement a custom sorting algorithm that partitions the data and sorts each part in parallel, then merges the results. This is something the standard `sort()` method doesn't support.

In summary, while the built-in `sort` works well when the entire dataset can fit into memory and when performance is not a bottleneck, you must resort to custom sorting algorithms in situations like external sorting or parallel processing when the built-in approach is impractical.